

SWITCHX STUDIO

Advanced Agentic AI Website Builder & Dev-Studio

TECHNICAL INTERVIEW ARCHITECTURE DECK

A Comprehensive Guide on Pipelines, System Logics, and Design Paradigms

AUTHOR / FOUNDER:

Mr. Shubham Sharma

Generated Autonomous System Documentation

Document Version: 2.1.0 (Gmail SMTP & Vision Dynamic update)

1. Executive Project Summary

SwitchX Studio is a next-generation web creation platform designed to empower developers and startups. Unlike standard drag-and-drop website builders (such as Wix or Squarespace), SwitchX is code-first and fully integrated with an Agentic AI engine. It translates natural language instructions directly into production-grade website layouts, compiles them on-the-fly, and serves them in a safe browser-based live emulator. In addition, it integrates multi-media generators, database mock streams, server logs, and a secure verification flow.

The Vision & The Founder

Founded and engineered solely by Mr. Shubham Sharma, SwitchX Studio was born out of a desire to eliminate the friction between high-level design ideas and physical source code. By utilizing state-of-the-art LLM reasoning, the platform acts as an autonomous pair-programmer that can build, test, audit, and deploy components directly.

Core Technology Stack

- * **Frontend Core:** React.js with Next.js Framework (App Router, Tailwind CSS, Framer Motion for premium micro-animations).
- * **Code Editor:** Monaco Editor integration (giving VS Code-style syntax highlighting, breadcrumbs, outline navigator, and minimap).
- * **AI Orchestration:** Google Gemini API (advanced text-and-vision model) with an automated local JSON fallback config to Groq/HuggingFace to guarantee 100% uptime.
- * **SMTP Auth Gateway:** Nodemailer core utilizing secure Gmail SMTP transport (powered by Google Account 2-Step Verification and 16-character App Passwords).
- * **Visual Generation:** Pollinations AI API Integration (delivering dynamic, high-fidelity mockups directly into the user stream).

2. The 4 Core Engineering Pipelines

To explain this project successfully to senior developers, you must describe the system in terms of data-flow pipelines. Here are the 4 main pipelines that make SwitchX work:

Pipeline 1: AI Code Generation & File Ingestion (Vibe-to-Code)

When a user asks the AI to build or edit a component, the text input goes through a structured pipeline to safely ingest, analyze, and render the resulting codebase:

[Logic/Trace Flow: Vibe-to-Code Pipeline Data Flow]



Pipeline 2: Live Emulation & Real-time Compilation

Unlike basic editors, SwitchX compiles code dynamically in the browser. When the frontend detects the presence of a generated website template (via a parsed HTML block), it prepares the emulator:

- * **HTML Assembly:** The app runs 'compileSectionsToHtml()', injecting a clean dark-aesthetic theme, custom fonts, particles.js graphics, and layout structures.
- * **Visual Sandbox:** The resulting HTML is rendered inside an <iframe>. This sandbox isolates execution so scripts do not interfere with the parent IDE page.
- * **State Sync:** The active tab 'activeSimulatorTab' allows users to toggle between 'Simulator' view, 'System Terminal' logs, and 'Telemetry' feeds.

Pipeline 3: Visual Asset & Image Generation

To make asset generation extremely responsive, we engineered an intelligent image pipeline. When the system detects visual requests (like 'generate a banner' or 'draw a wallpaper'):

- * **Keyword Matching:** A regex scanner matches trigger verbs: 'generate image', 'illustration of', 'logo of', 'render of', etc. This flags 'isVisualAsset = true'.

- * **Dynamic Dimensions:** The Gemini LLM evaluates the prompt to determine the optimal aspect ratio (e.g. 1920x1080 for desktop wallpapers, 1200x400 for banners, 1080x1920 for portrait cards) and quality flags.

- * **Pollinations AI:** The URL is dynamically constructed:

'https://image.pollinations.ai/p/[prompt]?width=W&height=H&seed=R&nologo=true&enhance=T'.

- * **UI Isolation:** To prevent the user from being interrupted, the Live Simulator is suppressed from opening automatically during image requests. Instead, the image is loaded inside a VisualLoadingWrapper component directly in the chat stream.

[Logic/Trace Flow: Visual Asset Pipeline]

```
[User Request: 'Create widescreen banner of neon city']
  |
  v
[demandsVisual = True] ---> Set isGeneratingImage(true)
  |
  v
[Gemini evaluates parameters] ---> returns: { imagePrompt: '...', width: 1200, height: 400, enhance:
true }
  |
  v
[Construct Pollinations URL] ---> https://image.pollinations.ai/p/...&width=1200&height=400
  |
  v
[Frontend Render] ---> Renders in Chat; suppresses Live Simulator auto-opening
```

Pipeline 4: SMTP Authentication & Gmail Gateway

A critical feature for user onboarding and account recovery is OTP verification. Since the developer does not want to buy a custom domain (which requires DNS TXT records and is heavily restricted by free-tier sandboxes like Resend), we set up a personal Gmail SMTP Gateway:

- * **The Gateway:** Nodemailer is configured to use 'service: gmail' with the sender credentials.

- * **Security Bypass:** Instead of using the account's master password, the system authenticates using a secure 16-character App Password generated via Google's 2-Step Verification portal.

- * **Universal Delivery:** This enables sending verification codes to ANY real email address, bypassing spam filters and domain verification hurdles.

3. 15 Critical Interview Q&As

Q1: What is the architectural role of Next.js in this project?

A: Next.js provides a unified full-stack architecture. The frontend utilizes React server/client components for rendering the IDE, while the App Router APIs (like '/api/send-otp' and '/api/builder') serve as lightweight, serverless API endpoints. This allows the frontend to securely interact with nodemailer and the Google Gemini API without exposing sensitive API keys to the browser.

Q2: Why did you transition from Resend to Gmail SMTP?

A: Resend's free tier requires a verified custom domain. Using their default 'onboarding@resend.dev' sandbox limits sending emails ONLY to the account owner's verified address. Since the goal was to allow any user to sign up and receive OTPs without buying a domain, Gmail SMTP was chosen. Google acts as a trusted outbound gateway, letting us send emails globally to any email provider (Gmail, Yahoo, Outlook) for free.

Q3: How does Gmail App Passwords fit into the security architecture?

A: Google blocked 'Less Secure Apps' in 2022. To connect programmatic clients like Nodemailer, we enable 2-Step Verification on the Google account and generate a 16-character App Password. This creates an isolated credential token specifically for SMTP sending. If this token is ever compromised, it can be revoked instantly without affecting the main Google account password.

Q4: How does the AI pipeline prevent rate-limiting or service interruptions?

A: The application implements a 'generateWithFallback' utility. When a request is sent to Google's Gemini API, if a rate-limit error (HTTP 429) or network timeout occurs, the backend automatically catches the exception and routes the payload to secondary providers (like Groq SDK or HuggingFace API) using pre-configured keys. This guarantees high availability (HA) and uptime.

Q5: How does the visual editor handle responsive layout generation?

A: The LLM model is instructed to output HTML structured with Tailwind utility classes. This provides flexibility because Tailwind's responsive prefixes (like md:, lg:, sm:) automatically adapt the compiled layout to different simulator viewport widths (Mobile, Tablet, Desktop) without writing complex custom CSS media queries.

Q6: How do you verify and parse the generated HTML inside the chat response?

A: In 'builderService.js', the text response from the LLM is scanned using a regular expression: `/`html\s*([\s\S]*?)\s*`/`. If a match is found, the inner HTML is extracted, the markdown block is stripped from the visible text chat bubble, and the clean HTML code is stored in the message object as 'generatedHtml' to be passed to the iframe compiler.

Q7: How did you design the user account recovery flow?

A: It is a 3-step security flow. First, the user enters their email. The backend generates a secure 6-digit OTP, saves it to server logs/database, and sends it via Gmail SMTP. Second, the user inputs the OTP; the frontend verifies it matches. Third, if verified, the system either displays their username (Forgot Username) or shows a new password field to update their credentials in the database (Forgot Password).

Q8: How does the application avoid duplicate email registrations during signup?

A: During signup, the registration handler performs a lookup search in our local JSON-based database for the requested email. If an existing user matches the email address, the backend blocks the creation and returns a 400 Bad Request status with a message: 'This email is already registered.'.

Q9: What is the benefit of using Pollinations AI for visual asset creation?

A: Pollinations AI is a free, fast, and unauthenticated image generation API that generates images from prompts on-the-fly. This allows the application to dynamically generate cinematic layouts, illustrations, and mockups directly using URL queries, removing the need for hosting expensive GPU clusters or storing bulky image files locally.

Q10: Explain why the Live Simulator is hidden during image creation.

A: Users found it disruptive when the live website emulator panel opened automatically while they were simply requesting images. To improve the user experience (UX), we updated 'MainStudio.js' to check: if a message contains a 'renderedImage' or has 'isVisualAsset' set to true, the simulator trigger button is suppressed. The image is neatly rendered inside the chat list instead, preserving focus.

Q11: How do you handle long-term memory for document uploads?

A: When a user uploads a dataset or text document, the text is extracted and stored in a database context buffer. When calling the Gemini model, this buffer is pre-pended to the active chat prompt. This gives the LLM context of the document files, allowing it to generate code and answer questions specifically tailored to the uploaded data.

Q12: How do you handle database persistence in the client workspace?

A: We use a file-system based database located under 'data/'. Users and mock tables are stored as raw JSON files. In Next.js serverless routes, we read and write these files using Node's 'fs' module. For client-side UI states and login sessions, we sync details directly to 'localStorage' under keys like 'switchx_active_user'.

Q13: How does the Monaco Editor integration improve developer experience?

A: Monaco Editor is the core editor that powers VS Code. By integrating it, SwitchX developers get access to features like multi-cursor editing, automatic brace matching, code folding, search-and-replace, and an outline panel. This bridges the gap between a simple playground and a professional IDE.

Q14: Explain the role of the System Terminal panel.

A: The System Terminal panel emulates a real Linux/Windows console shell. It captures server compiling logs, security audits, pipeline traces, and API responses. Having the terminal logs side-by-side with the simulator helps developers debug network requests and backend scripts in real-time.

Q15: What would be the next steps to scale SwitchX for enterprise use?

A: To scale this project, we would: 1. Transition the JSON file database to a managed SQL database (such as PostgreSQL with Prisma ORM). 2. Integrate a real Git hosting provider (like GitHub OAuth API) to push files directly to user repos. 3. Set up a secure containerized execution sandbox (like WebContainers or Docker) to compile TypeScript and run npm installs securely in the cloud.